

Train Less, Contribute More: A Neural-Architecture Search for Numerai’s Ender Target

Volodymyr Borysenko
University of California, Berkeley

October 2026

Abstract

We search for the neural network that best predicts Numerai’s Ender target (in v5.3 data, the 60-day horizon that Numerai pays on), judging models not by raw correlation but by benchmark model contribution (BMC): the signal a model adds beyond Numerai’s own LightGBM benchmark. Across 50 cross-validated runs in eight rounds we compare plain, wide, deep and residual multilayer perceptrons (MLPs), a TabM-style BatchEnsemble, learned per-bin feature embeddings, four loss functions and the main training hyperparameters. Architecture mattered far less than one variable: the amount of training. Training moves predictions steadily away from the benchmark, raising BMC, until it starts to destroy raw signal, and every knob we varied (epochs, learning rate, batch size, dropout, weight interpolation) lands on this single curve. The optimum is a fixed number of optimizer steps rather than epochs: on 613 eras never used during the search, the step-matched model reproduces the search-time correlation while the epoch-matched one over-trains. Adding a penalty on each era’s correlation with the benchmark raises BMC by a further 21% on those unseen eras. The final model, a plain 1024–512–256 MLP trained with a per-era correlation loss and this penalty, reaches BMC 0.0085 against the 60-day benchmark over the last 200 unseen eras, where a LightGBM baseline trained on the same data contributes almost nothing.

1 Introduction

Numerai runs a stock-market prediction tournament on obfuscated data: each week (an *era*) participants rank several thousand stocks from a few thousand anonymized features, and are paid for predictions that improve Numerai’s meta model. A model with high raw correlation that merely re-discovers what Numerai’s benchmark models already know contributes little. Benchmark model contribution (BMC) measures this directly as the correlation of a model’s predictions with the target after neutralizing them against the benchmark’s predictions [47, 50].

Gradient-boosted trees dominate tabular prediction [26], and the tournament’s benchmark is itself a LightGBM model. Neural networks therefore have a natural advantage for BMC: their inductive bias differs from that of trees, so even at lower raw correlation they may carry more *unique* signal. This paper asks which neural architecture, loss and training recipe maximize that unique signal.

Our contributions are:

1. A controlled architecture search, one variable at a time with seed-averaged confirmation, showing that a plain MLP beats residual, ensemble and embedding-based alternatives on BMC.
2. Evidence that BMC is governed by the *amount* of training, and that this trade-off has a single interior optimum shared by all training knobs.
3. A confirmation on disjoint data showing that the optimum is a fixed number of optimizer steps, so the epoch count must shrink as the training set grows.

4. A benchmark-penalized correlation loss that raises BMC by about a fifth on unseen eras, using the benchmark’s honest out-of-sample predictions only during training.

2 Related work

Machine learning for cross-sectional returns. Neural networks and trees improve out-of-sample return prediction over linear models through nonlinear interactions, and shallow networks outperform deeper ones [29]; later work learns factor structure [30] or a stochastic discount factor with moment-based losses [16], and finds machine-learning profits concentrated in hard-to-arbitrage stocks [3]. Whether model complexity is a virtue [36] remains debated [20, 49]. Losses closer to the trading objective, such as ranking [44, 51], classification [5], Sharpe-ratio [64] and correlation losses [48], often beat squared error. Hedging unpriced risk [18] motivates the feature neutralization common on Numerai [11, 13]; forecast combination [53], auxiliary targets [46], dynamic ensembling under regime change [61] and era boosting [54] are other recurring tools. Repeated configuration search inflates backtests [2, 6, 7]; shrinkage is a classic remedy [39].

Tabular deep learning. Recent work revisits MLP, ResNet and transformer baselines [23], numerical feature embeddings [24], parameter-efficient ensembles [25], well-tuned default MLPs [33] and heavily regularized plain networks [35]. Attention-based models [58] and tabular foundation models [27, 28, 32] do not yet scale to millions of rows with hundreds of features. Weight averaging [34], deep ensembles [40] and model soups [62] reduce variance.

Training dynamics and differential equations. Gradient flow stopped at time t behaves like ridge regression with penalty $\lambda = 1/t$ [1]; finite learning rates add an implicit gradient penalty whose strength scales with the learning rate and, for stochastic gradients, inversely with batch size [8, 41, 57]. Decay towards the initialization is the explicit counterpart of early stopping [43], and noise injection acts as Tikhonov or heat-equation smoothing [9, 14, 15, 59]. Continuous-depth networks [17, 19, 21, 31, 37, 56] and their stochastic and controlled variants [38, 42, 45] have been applied to finance [22, 63], building on stochastic-calculus models of prices [10], but we found no application to static cross-sectional ranking.

Number theory. Its one documented use relevant here is quasi-Monte Carlo search: low-discrepancy sequences cover a hyperparameter space more evenly than random draws [12, 55]. Random Fourier features [4, 52] and feature hashing [60] are related tools that do not fit dense, low-cardinality features.

3 Experimental setup

Data. Numerai v5.3, `medium` feature set (780 features, each an integer bin 0–4), target column `target`. In v5.3 this column is identical to `target_ender_60` on every row (it matches `target_ender_20` on only 47%), following Numerai’s switch to 60-day scoring in round 1343 [50]. We trained on it throughout; we initially took it for the 20-day target, a mistake that affected only the choice of benchmark during the search (see Metrics). Because experiments ran on a 16 GB laptop, the search used every fourth era of the training and validation data (307 eras, 1.7M rows; *scout data*). The confirmation step used every second era at offset one (613 eras, 3.4M rows; *scale data*), which shares no eras with the scout data.

Validation. Five expanding-window folds over eras with a 13-era embargo. The first fold has no training data, so out-of-fold (OOF) predictions cover the last four folds. Every network trains

Architecture (MSE, 4 epochs)	corr	BMC	BMC last 200	corr w/ benchmark
Wide MLP 1024–512–256	0.0259	0.0061	0.0062	0.404
MLP 256–128	0.0249	0.0053	0.0052	0.402
LightGBM baseline	0.0277	0.0042	0.0042	0.485
Residual MLP (2 blocks, width 256)	0.0187	0.0036	0.0037	0.319
TabM BatchEnsemble ($k = 8$)	0.0213	0.0026	0.0026	0.393
MLP with per-bin embeddings	0.0157	0.0012	0.0006	0.312

Table 1: Round-1 architecture scout on scout data, single seeds; BMC against the ender20 benchmark (search metric).

for a fixed number of epochs; there is no early stopping, so no validation information leaks into OOF scores.

Metrics. The primary metric is mean BMC over the last 200 OOF eras. During the architecture search (rounds 1–6) it was computed against the official `v53_lgbm_ender20` predictions; once we found that the target is the 60-day Ender, every saved prediction was re-scored against `v53_lgbm_ender60`, the benchmark that matches the target, and rounds 7–8 and the final model use that metric. The ranking of the search finalists is the same under both (§4.5). We also report BMC over all OOF eras, raw per-era correlation (corr), and the mean correlation between a model’s predictions and the benchmark’s.

Models. All networks share one PyTorch implementation trained on Apple-silicon GPUs. Inputs are scaled to $(x - 2)/2$ unless embeddings are used. The MSE loss uses random batches of 4,096 rows. The correlation loss uses batches of four whole eras and minimizes the negative mean per-era Pearson correlation

$$\mathcal{L}_{\text{corr}} = -\frac{1}{|E|} \sum_{e \in E} \frac{\langle \hat{y}_e - \bar{\hat{y}}, y_e - \bar{y} \rangle}{\|\hat{y}_e - \bar{\hat{y}}\| \|y_e - \bar{y}\|}. \quad (1)$$

Optimization uses AdamW with a cosine schedule and 5% warm-up. The baseline is LightGBM with 2,000 trees and 31 leaves on identical data, features and folds.

Protocol. Rounds of four to eight configurations, each changing one variable from the current leader. Gaps smaller than seed noise were re-checked with three-seed averages before being trusted. The search stopped after two consecutive rounds without a confirmed improvement.

4 Results

4.1 A plain MLP beats more elaborate architectures

Under an identical recipe (MSE, four epochs) the plain MLP beats LightGBM on BMC despite lower raw correlation (Table 1, Figure 1). Its predictions correlate only 0.40 with the benchmark, against 0.49 for LightGBM, so more of its signal is new. Every attempt to add structure did worse. Learned per-bin embeddings quadruple the input width and overfit; with only five bin values a piecewise-linear embedding [24] reduces to one-hot encoding. Residual blocks and TabM ensembling also lost, as did a deeper (512–256–128) and a wider (2048–1024–512) network in later rounds.

4.2 Training amount governs contribution

Training directly on per-era correlation lowers the benchmark correlation of predictions far more than MSE does, but at 20 epochs it overfits badly: training-era correlation reached 0.24 against

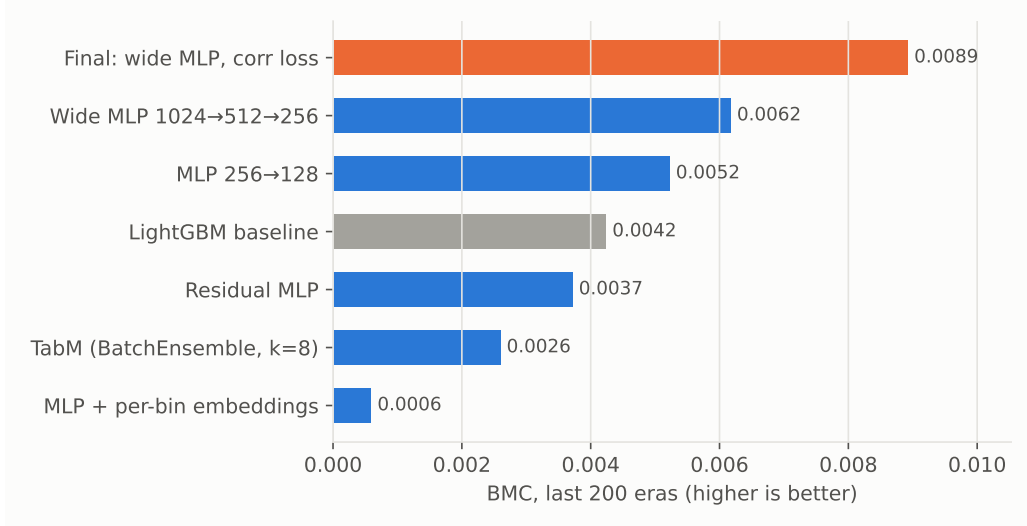


Figure 1: BMC over the last 200 OOF eras. Blue bars share the untuned round-1 recipe; gray is the LightGBM baseline; orange is the final tuned model (correlation loss, learning rate 2×10^{-3} , six epochs, three-seed average).

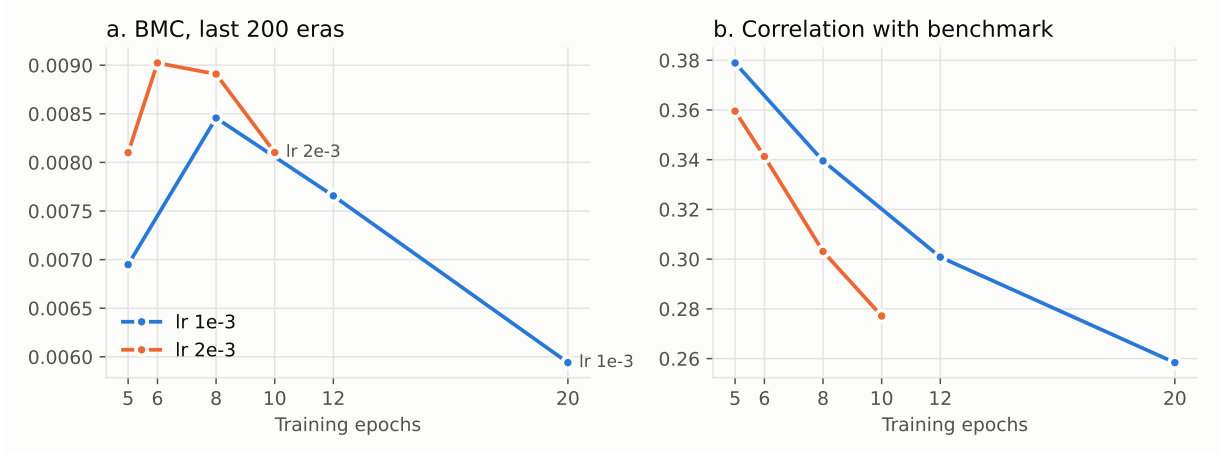


Figure 2: Correlation-loss MLP (256–128), single seeds. (a) BMC peaks at a training length that depends on the learning rate. (b) Correlation with the benchmark falls the longer the network trains. The point at learning rate 2×10^{-3} and eight epochs is a lucky seed; its three-seed average is 0.0083.

0.018 out of fold. More dropout and weight decay, or an added MSE term, did not fix this as well as simply training less.

Figure 2 shows the mechanism. As the network trains, its predictions drift steadily away from the benchmark (panel b), which raises BMC, while raw correlation holds roughly flat and then falls. BMC therefore peaks just before raw correlation collapses (panel a). Every knob that changes the amount of training lands on this curve: the optimal epoch count moves from 8 to 6 when the learning rate doubles; a learning rate of 5×10^{-4} under-trains while 3×10^{-3} over-trains; eight eras per batch halves the number of updates and under-trains; and extra dropout or input dropout at the optimal length behaves like less training, pulling predictions back towards the benchmark. Across all scout runs, BMC rises as benchmark correlation falls until raw signal is lost (Figure 3).

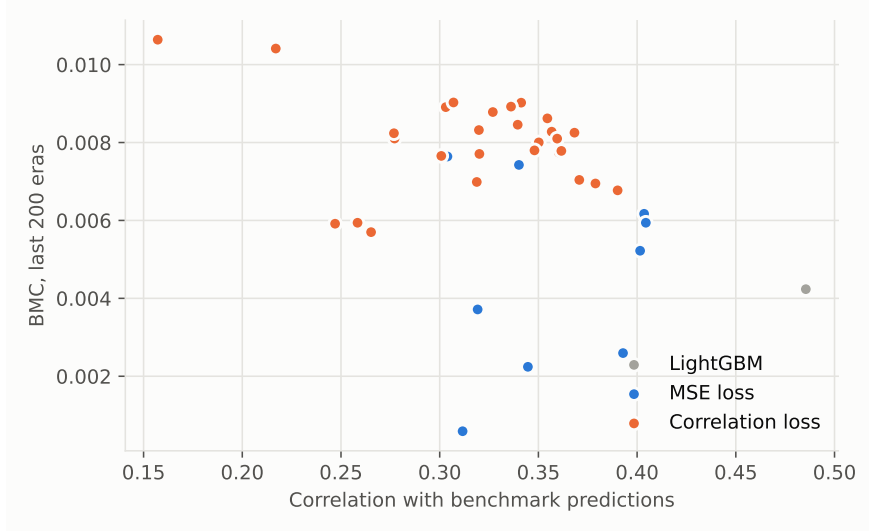


Figure 3: All scout runs: correlation with the benchmark against BMC. Lower benchmark correlation buys contribution until raw signal collapses.

Model (scout data, 3 seeds)	corr	BMC	BMC last 200	Sharpe	w/ bench.
Wide 1024–512–256, corr loss, lr 2e-3, 6 ep.	0.0260	0.0088	0.0089	0.71	0.336
Narrow 256–128, corr loss, lr 2e-3, 6 ep.	0.0262	0.0085	0.0086	0.71	0.355
Narrow 256–128, corr loss, lr 1e-3, 8 ep.	0.0261	0.0082	0.0083	0.68	0.357
Rank average of wide and narrow	0.0263	0.0087	0.0088	0.71	–

Table 2: Seed-averaged finalists on scout data; BMC against the ender20 benchmark (search metric).

4.3 Seed noise and the confirmed winner

Single-seed results vary by up to ± 0.0006 BMC: one run at learning rate 2×10^{-3} scored 0.0089, but its three-seed average was 0.0083. We therefore judged finalists on three-seed averages (Table 2). The wide and narrow winners’ predictions correlate 0.96, so averaging them adds nothing. Width helps only once training is short enough to stop the network from memorizing. Figure 4 shows that the winner’s advantage accumulates steadily across eras rather than coming from a few outliers.

4.4 Confirmation on unseen eras

We retrained the finalists on the scale data: 613 eras never used during the search, with twice the training rows per fold (Table 4). Run unchanged, the six-epoch winner dropped to 0.0056, with exactly the signature of over-training seen in Figure 2: lower raw correlation and lower benchmark correlation (0.28 against 0.34 on scout data). With twice the rows per epoch, six epochs means twice as many optimizer steps. Matching the step count instead (three epochs) reproduces the scout raw correlation almost exactly (0.0262 against 0.0260) and reaches BMC 0.0095 on data the search never touched; BMC then falls monotonically as steps increase (Figure 5). The optimal training amount is therefore a fixed number of optimizer steps, not epochs, consistent with the continuous-time view in which stopping time sets the effective regularization [1]. A squared-error network trained for eight epochs on the scale data over-trains the same way (BMC 0.0027, benchmark correlation 0.23).

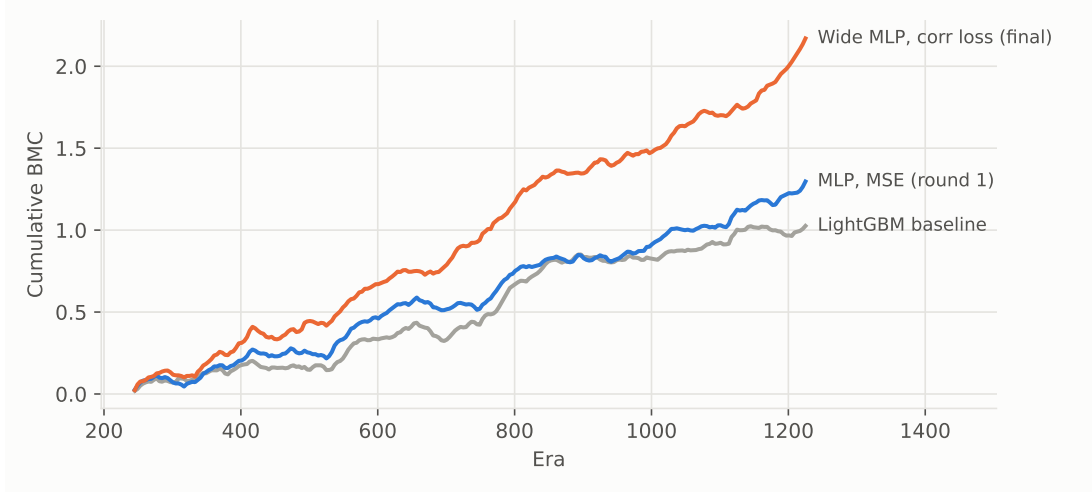


Figure 4: Cumulative per-era BMC on the same 246 scout OOF eras.

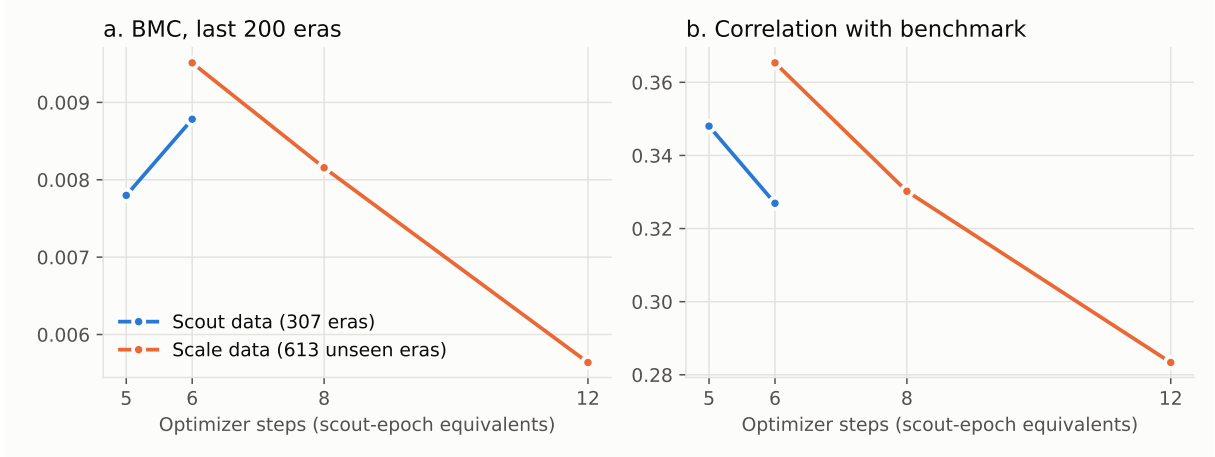


Figure 5: Wide correlation-loss MLP on scout data and on the disjoint scale data, plotted against training amount in optimizer steps (one unit = one scout epoch). Scale points are three-seed averages; scout points are single seeds. Matching the step count transfers the optimum to unseen eras.

4.5 Scoring against the matching benchmark

Re-scoring the saved out-of-fold predictions against the 60-day benchmark `v53_lgbm_ender60`, which matches the target, preserves the ranking and widens the gap (Table 3): the LightGBM baseline adds almost nothing beyond the 60-day benchmark, while the wide correlation-loss network still contributes 0.0064.

4.6 Penalizing the benchmark

The training-amount curve suggests a direct route to uniqueness: tell the network what the benchmark already knows. Because the benchmark’s predictions on training eras are honest out-of-sample forecasts (their per-era correlation with the target is 0.044 on training eras and 0.048 on validation eras), they can be used in the loss without leakage. We add a penalty on each era’s squared correlation with the benchmark,

$$\mathcal{L} = \mathcal{L}_{\text{corr}} + \lambda \frac{1}{|E|} \sum_{e \in E} \text{corr}(\hat{y}_e, b_e)^2, \quad (2)$$

where b_e is the benchmark’s prediction; the benchmark is never an input to the network. Table 5 shows that the penalty raises BMC by 23% on scout data and 21% on the unseen scale eras, at

Model (scout data)	corr	BMC	BMC last 200	corr w/ benchmark
Wide MLP, corr loss, 3 seeds	0.0260	0.0068	0.0064	0.357
Narrow MLP, corr loss, lr 2e-3, 3 seeds	0.0262	0.0065	0.0061	0.374
Narrow MLP, corr loss, lr 1e-3, 3 seeds	0.0261	0.0062	0.0057	0.376
MLP, MSE, 8 epochs	0.0244	0.0049	0.0046	0.366
MLP, MSE, 4 epochs	0.0249	0.0030	0.0023	0.423
LightGBM baseline	0.0277	0.0010	0.0002	0.518

Table 3: Out-of-fold predictions re-scored on the 60-day payout target against the 60-day LightGBM benchmark. All models were trained on `target` (= `target_ender_60`).

Model (3 seeds)	corr	BMC	BMC last 200	corr w/ benchmark
Wide, 6 epochs, scout data (reference)	0.0260	0.0088	0.0089	0.336
Wide, 3 epochs ($1.0\times$ steps)	0.0262	0.0083	0.0095	0.365
Wide, 4 epochs ($1.33\times$ steps)	0.0237	0.0074	0.0082	0.330
Wide, 6 epochs ($2.0\times$ steps)	0.0206	0.0065	0.0056	0.283
Narrow, 6 epochs ($2.0\times$ steps)	0.0220	0.0068	0.0067	0.308
Wide, MSE, 8 epochs	0.0164	0.0049	0.0027	0.233

Table 4: Confirmation on 613 eras disjoint from the search; BMC against the ender20 benchmark (search metric). The last 200 eras of the scale data are more recent than those of the scout data, so absolute levels are not directly comparable.

the cost of raw correlation. Penalizing the benchmark the model is scored against works slightly better than penalizing the 20-day one, and $\lambda = 1$ buys no more BMC than $\lambda = 0.3$ while losing raw signal. Directly maximizing BMC (correlation of the prediction after removing its benchmark component within each era) helped little: the network remained free to stay close to the benchmark (correlation 0.31). Weight interpolation towards the initialization, $\theta_0 + \alpha(\theta_T - \theta_0)$ [1], behaved exactly like the amount of training ($\alpha = 0.8$ under-trains, $\alpha = 1.2$ slightly over-trains) and added little beyond it.

Blending trades contribution for correlation. Because uniqueness costs raw correlation, we also tested rank-averaging the penalized network with a tree model (Table 6). Numerai passes the benchmark’s live predictions to uploaded models, so the benchmark itself is a free blending partner. A 25% blend raises correlation by 62% and its Sharpe ratio from 1.41 to 2.23 for a 19% loss of BMC; which side of this trade to take depends on how a participant weights correlation against contribution.

5 Discussion

Why does less capacity win? Numerai’s per-era signal is weak, with correlations around 0.02, so spare capacity is spent fitting era-specific noise. The architectures that lost all add capacity or flexibility; the winning changes all limit how far the network moves from its initialization. This mirrors the shallow-network result for U.S. equities [29] and the strength of regularized plain MLPs on tabular data [35].

Why does a correlation loss help bmc? Squared error rewards predicting the target’s overall level and spread, which a tree benchmark already does well. A per-era Pearson loss rewards only within-era ranking, the quantity Numerai scores, and in practice yields predictions far less similar to the benchmark’s.

Model (3 seeds, BMC vs. ender60 benchmark)	corr	BMC	last 200	w/ bench.
<i>Scout data</i>				
Wide MLP, corr loss, no penalty	0.0260	0.0068	0.0064	0.357
+ penalty vs. ender60 benchmark, $\lambda = 0.3$	0.0215	0.0081	0.0079	0.249
+ penalty vs. ender60 benchmark, $\lambda = 1$	0.0184	0.0081	0.0080	0.191
+ penalty vs. ender20 benchmark, $\lambda = 0.3$	0.0220	0.0077	0.0075	0.265
<i>Scale data (613 unseen eras, step-matched)</i>				
Wide MLP, corr loss, no penalty	0.0262	0.0063	0.0070	0.384
+ penalty vs. ender60 benchmark, $\lambda = 0.3$	0.0208	0.0074	0.0085	0.255

Table 5: Benchmark penalty, three-seed averages.

Blend partner (scout data)	weight	corr	corr Sharpe	BMC last 200
None (penalized network)	0	0.0215	1.41	0.0079
Official benchmark	0.25	0.0348	2.23	0.0064
Official benchmark	0.50	0.0432	2.53	0.0043
LightGBM baseline	0.25	0.0268	1.74	0.0066
LightGBM baseline	0.50	0.0293	1.85	0.0047

Table 6: Rank-average blends of the penalized network with a tree model; BMC against the ender60 benchmark.

A practical recipe. For a new target or feature set, tune the amount of training first, in optimizer steps, and pick the BMC peak before adding architecture. When the training set grows, scale the epoch count down so the number of steps stays at the optimum.

Limitations. We used the `medium` feature set only; the 3,555-feature set did not fit in memory. The official BMC uses a stake-weighted ensemble of benchmark models [50]; we scored against the single 60-day LightGBM benchmark. Finalists were averaged over three seeds, so differences below about 0.0003 should be read as ties. Fifty configurations were tried on the same folds; a deflated Sharpe ratio [6] would temper the best scout result, which is one reason we confirmed on disjoint eras. No live-tournament results are included yet.

6 Conclusion

On Numerai’s Ender target the best neural network is not an exotic one. A plain three-layer MLP with a per-era correlation loss, trained briefly and told what the benchmark already knows, contributes signal that a LightGBM model trained on the same data does not. Architecture mattered less than the amount of training, which trades raw signal for uniqueness along a single curve. That amount is a fixed number of optimizer steps: holding it fixed transferred the optimum to unseen eras. Next steps follow from the literature: a quasi-Monte Carlo search over the learning-rate, step-count and penalty interaction [12], auxiliary targets as extra heads [46], partial feature neutralization [13], and the full 3,555-feature set.

Reproducibility. Code, configurations, figures and this paper are in <https://github.com/VolodymyrLinuxovich/example-scripts>, branch `nn-arch-ender20`, directory `numerai/agents/experiments/nn_arch_ender20`.

References

- [1] Alnur Ali, J. Zico Kolter, and Ryan J. Tibshirani. A continuous-time view of early stopping for least squares. In *AISTATS*, 2019.
- [2] Hamid Arian, Daniel Norouzi, and Luis Seco. Backtest overfitting in the machine learning era: A comparison of out-of-sample testing methods in a synthetic controlled environment. *Knowledge-Based Systems*, 2024. URL <https://www.ssrn.com/abstract=4778909>.
- [3] Doron Avramov, Si Cheng, and Lior Metzker. Machine learning vs. economic restrictions: Evidence from stock return predictability. *Management Science*, 2023. doi: 10.1287/mnsc.2022.4449.
- [4] Haim Avron, Vikas Sindhwani, Jiyan Yang, and Michael W. Mahoney. Quasi-Monte Carlo feature maps for shift-invariant kernels. 2016.
- [5] Yang Bai and Kuntara Pukthuanthong. Machine learning classification and portfolio construction: Does the loss function matter?, 2021.
- [6] David H. Bailey and Marcos López de Prado. The deflated sharpe ratio: Correcting for selection bias, backtest overfitting and non-normality. *Journal of Portfolio Management*, 2014. URL https://papers.ssrn.com/sol3/papers.cfm?abstract_id=2460551.
- [7] David H. Bailey, Jonathan Borwein, Marcos López de Prado, and Qiji Jim Zhu. The probability of backtest overfitting. *Journal of Computational Finance*, 2017. URL <https://escholarship.org/uc/item/4w1110bb>.
- [8] David G. T. Barrett and Benoit Dherin. Implicit gradient regularization. In *ICLR*, 2021.
- [9] Chris M. Bishop. Training with noise is equivalent to Tikhonov regularization. *Neural Computation*, 7(1):108–116, 1995. doi: 10.1162/neco.1995.7.1.108.
- [10] Fischer Black and Myron Scholes. The pricing of options and corporate liabilities. *Journal of Political Economy*, 1973. doi: 10.1086/260062.
- [11] bobyfisch. An introduction to feature neutralization / exposure. Numerai forum, <https://forum.numer.ai/t/an-introduction-to-feature-neutralization-exposure/4955>, 2022.
- [12] Olivier Bousquet, Sylvain Gelly, Karol Kurach, Olivier Teytaud, and Damien Vincent. Critical hyper-parameters: No random, no cry, 2017.
- [13] by256. Feature neutralization increases bias and reduces variance. Numerai forum, <https://forum.numer.ai/t/feature-neutralization-increases-bias-and-reduces-variance/7486>, 2024.
- [14] Ryan Campbell, Chris Finlay, and Adam M. Oberman. Deterministic Gaussian averaged neural networks, 2020.
- [15] Alexander Camuto, Matthew Willetts, Umut Şimşekli, Stephen Roberts, and Chris Holmes. Explicit regularisation in Gaussian noise injections. In *NeurIPS*, 2020.
- [16] Luyang Chen, Markus Pelger, and Jason Zhu. Deep learning in asset pricing. *Management Science*, 2024. doi: 10.1287/mnsc.2023.4695. URL <https://arxiv.org/abs/1904.00745>.
- [17] Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations. In *NeurIPS*, 2018.

- [18] Kent Daniel, Lira Mota, Simon Rottke, and Tano Santos. The cross-section of risk and returns. *Review of Financial Studies*, 33(5):1927–1979, 2020.
- [19] Emilien Dupont, Arnaud Doucet, and Yee Whye Teh. Augmented neural ODEs. In *NeurIPS*, 2019.
- [20] Hasan Fallahgoul. High-dimensional learning in finance, 2025.
- [21] Chris Finlay, Jörn-Henrik Jacobsen, Levon Nurbekyan, and Adam M. Oberman. How to train your neural ODE: The world of Jacobian and kinetic regularization. In *ICML*, 2020.
- [22] Patryk Gierjatowicz, Marc Sabate-Vidales, David Šiška, Lukasz Szpruch, and Žan Žurič. Robust pricing and hedging via neural SDEs, 2020.
- [23] Yury Gorishniy, Ivan Rubachev, Valentin Khrulkov, and Artem Babenko. Revisiting deep learning models for tabular data. In *NeurIPS*, 2021.
- [24] Yury Gorishniy, Ivan Rubachev, and Artem Babenko. On embeddings for numerical features in tabular deep learning. In *NeurIPS*, 2022.
- [25] Yury Gorishniy, Akim Kotelnikov, and Artem Babenko. TabM: Advancing tabular deep learning with parameter-efficient ensembling. In *ICLR*, 2025.
- [26] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. Why do tree-based models still outperform deep learning on tabular data? In *NeurIPS Datasets and Benchmarks*, 2022.
- [27] Léo Grinsztajn, Klemens Flöge, et al. TabPFN-2.5: Advancing the state of the art in tabular foundation models, 2025.
- [28] Léo Grinsztajn, Klemens Flöge, et al. TabPFN-3: Technical report, 2026.
- [29] Shihao Gu, Bryan Kelly, and Dacheng Xiu. Empirical asset pricing via machine learning. *Review of Financial Studies*, 2020. URL https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3159577.
- [30] Shihao Gu, Bryan Kelly, and Dacheng Xiu. Autoencoder asset pricing models. *Journal of Econometrics*, 2021. URL <https://www.semanticscholar.org/paper/Autoencoder-Asset-Pricing-Models-Gu-Kelly/5953bcc383c3ceb866b8ef7f7c16d166e5f2ab4d>.
- [31] Eldad Haber and Lars Ruthotto. Stable architectures for deep neural networks. *Inverse Problems*, 2017.
- [32] Noah Hollmann, Samuel Müller, Lennart Purucker, et al. Accurate predictions on small data with a tabular foundation model. *Nature*, 637:319–326, 2025. doi: 10.1038/s41586-024-08328-6.
- [33] David Holzmüller, Léo Grinsztajn, and Ingo Steinwart. Better by default: Strong pre-tuned MLPs and boosted trees on tabular data. In *NeurIPS*, 2024.
- [34] Pavel Izmailov, Dmitrii Podoprikin, Timur Garipov, Dmitry Vetrov, and Andrew Gordon Wilson. Averaging weights leads to wider optima and better generalization. In *UAI*, 2018.
- [35] Arlind Kadra, Marius Lindauer, Frank Hutter, and Josif Grabocka. Well-tuned simple nets excel on tabular datasets. In *NeurIPS*, 2021.
- [36] Bryan Kelly, Semyon Malamud, and Kangying Zhou. The virtue of complexity in return prediction. *Journal of Finance*, 2024. doi: 10.1111/jofi.13298.

- [37] Jacob Kelly, Jesse Bettencourt, Matthew James Johnson, and David Duvenaud. Learning differential equations that are easy to solve. In *NeurIPS*, 2020.
- [38] Patrick Kidger, James Morrill, James Foster, and Terry Lyons. Neural controlled differential equations for irregular time series. In *NeurIPS*, 2020.
- [39] Serhiy Kozak, Stefan Nagel, and Shrihari Santosh. Shrinking the cross-section. *Journal of Financial Economics*, 135(2):271–292, 2020.
- [40] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *NIPS*, 2017.
- [41] Qianxiao Li, Cheng Tai, and Weinan E. Stochastic modified equations and adaptive stochastic gradient algorithms. In *ICML*, 2017.
- [42] Xuechen Li, Ting-Kam Leonard Wong, Ricky T. Q. Chen, and David Duvenaud. Scalable gradients for stochastic differential equations. In *AISTATS*, 2020.
- [43] Xuhong Li, Yves Grandvalet, and Franck Davoine. Explicit inductive bias for transfer learning with convolutional networks. In *ICML*, 2018.
- [44] Yan Lin, Yihong Su, and Yi Yang. LambdaRankIC: Directly optimizing rank IC for financial prediction, 2026.
- [45] Xuanqing Liu, Tesi Xiao, Si Si, Qin Cao, Sanjiv Kumar, and Cho-Jui Hsieh. Neural SDE: Stabilizing neural ODE networks with stochastic noise, 2019.
- [46] master_key. New targets for the tournament. Numerai forum, <https://forum.numer.ai/t/new-targets-for-the-tournament/5842>, 2022.
- [47] master_key. Benchmark models. Numerai forum, <https://forum.numer.ai/t/benchmark-models/6754>, 2023.
- [48] mdo. Custom loss functions for XGBoost using PyTorch. Numerai forum, <https://forum.numer.ai/t/custom-loss-functions-for-xgboost-using-pytorch/960>, 2020.
- [49] Stefan Nagel. Seemingly virtuous complexity in return prediction. Working Paper 34104, National Bureau of Economic Research, 2025. URL <https://www.nber.org/papers/w34104>.
- [50] Numerai. Scoring. <https://docs.numer.ai/numerai-tournament/scoring>, 2026. Accessed 2026-10-04.
- [51] Daniel Poh, Bryan Lim, Stefan Zohren, and Stephen Roberts. Building cross-sectional systematic strategies by learning to rank, 2020.
- [52] Ali Rahimi and Benjamin Recht. Random features for large-scale kernel machines. In *NIPS*, 2007. URL <https://papers.nips.cc/paper/3182-random-features-for-large-scale-kernel-machines>.
- [53] David E. Rapach, Jack K. Strauss, and Guofu Zhou. Out-of-sample equity premium prediction: Combination forecasts and links to the real economy. *Review of Financial Studies*, 2010. URL https://papers.ssrn.com/sol3/papers.cfm?abstract_id=1257858.
- [54] richai. Era boosted models. Numerai forum, <https://forum.numer.ai/t/era-boosted-models/189>, 2020.
- [55] Pamphile T. Roy, Art B. Owen, Maximilian Balandat, and Matt Haberland. Quasi-Monte Carlo methods in Python. *Journal of Open Source Software*, 8(84):5309, 2023. doi: 10.21105/joss.05309.

- [56] Lars Ruthotto and Eldad Haber. Deep neural networks motivated by partial differential equations. *Journal of Mathematical Imaging and Vision*, 2020.
- [57] Samuel L. Smith, Benoit Dherin, David G. T. Barrett, and Soham De. On the origin of implicit regularization in stochastic gradient descent. In *ICLR*, 2021.
- [58] Gowthami Somepalli, Micah Goldblum, Avi Schwarzschild, C. Bayan Bruss, and Tom Goldstein. SAINT: Improved neural networks for tabular data via row attention and contrastive pre-training, 2021.
- [59] Tangjun Wang, Chenglong Bao, and Zuoqiang Shi. Convection-diffusion equation: A theoretically certified framework for neural networks, 2024.
- [60] Kilian Weinberger, Anirban Dasgupta, John Langford, Alex Smola, and Josh Attenberg. Feature hashing for large scale multitask learning. In *ICML*, 2009.
- [61] Thomas Wong and Mauricio Barahona. Online learning techniques for prediction of temporal tabular datasets with regime changes, 2023.
- [62] Mitchell Wortsman et al. Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *ICML*, 2022.
- [63] Luxuan Yang, Ting Gao, Yubin Lu, Jinqiao Duan, and Tao Liu. Neural network stochastic differential equation models with applications to financial data forecasting, 2021.
- [64] Zihao Zhang, Stefan Zohren, and Stephen Roberts. Deep learning for portfolio optimization, 2020.

A Full experiment log

Every configuration that ran, by round. Single seeds unless the configuration says otherwise; bold marks each round’s best BMC over the last 200 eras.

Baselines

Run	Configuration	corr	BMC	last 200	w/ bench.
r0_baseline_small_lgbm	LightGBM, 2,000 trees	0.0277	0.0042	0.0042	0.485
r0_smoke_mlp	MLP 256–128 · MSE · 1 ep · lr 0.001	0.0188	0.0024	0.0022	0.345

Round 1 · architecture

Run	Configuration	corr	BMC	last 200	w/ bench.
r1_mlp_wide_mse	MLP 1024–512–256 · MSE · 4 ep · lr 0.001 · drop 0.2	0.0259	0.0061	0.0062	0.404
r1_mlp_corr	MLP 256–128 · corr loss · 20 ep · lr 0.001	0.0184	0.0055	0.0059	0.258
r1_mlp_mse	MLP 256–128 · MSE · 4 ep · lr 0.001	0.0249	0.0053	0.0052	0.402
r1_resmlp_mse	Residual MLP w256 · MSE · 4 ep · lr 0.001	0.0187	0.0036	0.0037	0.319
r1_tabm_mse	TabM 256–256 · MSE · 4 ep · lr 0.001	0.0213	0.0026	0.0026	0.393
r1_mlp_embed_mse	MLP 256–128 · MSE · 4 ep · lr 0.001 · embeddings	0.0157	0.0012	0.0006	0.312

Round 2 · training length

Run	Configuration	corr	BMC	last 200	w/ bench.
r2_corr_ep8	MLP 256–128 · corr loss · 8 ep · lr 0.001	0.0253	0.0083	0.0085	0.340
r2_corr_epb1	MLP 256–128 · corr loss · 5 ep · lr 0.001 · 1 era/batch	0.0254	0.0078	0.0080	0.350
r2_wide_ep8	MLP 1024–512–256 · MSE · 8 ep · lr 0.001 · drop 0.2	0.0229	0.0073	0.0076	0.304
r2_mse_ep8	MLP 256–128 · MSE · 8 ep · lr 0.001	0.0244	0.0071	0.0074	0.340
r2_corr_reg	MLP 256–128 · corr loss · 20 ep · lr 0.001 · drop 0.3 · wd 0.001	0.0230	0.0068	0.0070	0.319
r2_wider_mse	MLP 2048–1024–512 · MSE · 4 ep · lr 0.001 · drop 0.3	0.0255	0.0057	0.0059	0.404
r2_wide_corr	MLP 1024–512–256 · corr loss · 20 ep · lr 0.001 · drop 0.2	0.0183	0.0059	0.0059	0.247
r2_corr_mse	MLP 256–128 · corr+MSE · 20 ep · lr 0.001	0.0184	0.0052	0.0057	0.265

Round 3 · refine

Run	Configuration	corr	BMC	last 200	w/ bench.
r3_corr_ep8_seeds3	MLP 256–128 · corr loss · 8 ep · lr 0.001 · 3 seeds	0.0261	0.0082	0.0083	0.357
r3_wide_corr_ep6	MLP 1024–512–256 · corr loss · 6 ep · lr 0.001 · drop 0.2	0.0261	0.0079	0.0077	0.360
r3_corr_ep12	MLP 256–128 · corr loss · 12 ep · lr 0.001	0.0225	0.0073	0.0077	0.301
r3_corr_ep5	MLP 256–128 · corr loss · 5 ep · lr 0.001	0.0253	0.0067	0.0069	0.379
r3_corr_ep8_drop03	MLP 256–128 · corr loss · 8 ep · lr 0.001 · drop 0.3	0.0260	0.0067	0.0068	0.390

Round 4 · new knobs

Run	Configuration	corr	BMC	last 200	w/ bench.
r4_lr2e3	MLP 256–128 · corr loss · 8 ep · lr 0.002	0.0237	0.0084	0.0089	0.303
r4_indrop01	MLP 256–128 · corr loss · 8 ep · lr 0.001 · in-drop 0.1	0.0263	0.0079	0.0083	0.368
r4_epb8	MLP 256–128 · corr loss · 8 ep · lr 0.001 · 8 era/batch	0.0250	0.0072	0.0078	0.362
r4_deep	MLP 512–256–128 · corr loss · 8 ep · lr 0.001	0.0241	0.0076	0.0077	0.320
r4_lr5e4	MLP 256–128 · corr loss · 8 ep · lr 0.0005	0.0252	0.0068	0.0070	0.371

Round 5 · lr 2e-3

Run	Configuration	corr	BMC	last 200	w/ bench.
r5_lr2e3_ep6	MLP 256–128 · corr loss · 6 ep · lr 0.002	0.0257	0.0087	0.0090	0.341
r5_wide_lr2e3_ep6	MLP 1024–512–256 · corr loss · 6 ep · lr 0.002 · drop 0.2	0.0256	0.0088	0.0088	0.327
r5_lr2e3_seeds3	MLP 256–128 · corr loss · 8 ep · lr 0.002 · 3 seeds	0.0242	0.0080	0.0083	0.320
r5_lr3e3	MLP 256–128 · corr loss · 8 ep · lr 0.003	0.0217	0.0078	0.0082	0.277
r5_lr2e3_ep10	MLP 256–128 · corr loss · 10 ep · lr 0.002	0.0216	0.0075	0.0081	0.277

Round 6 · seed check

Run	Configuration	corr	BMC	last 200	w/ bench.
r6_wide_lr2e3_ep6_seeds3	MLP 1024–512–256 · corr loss · 6 ep · lr 0.002 · drop 0.2 · 3 seeds	0.0260	0.0088	0.0089	0.336
r6_lr2e3_ep6_seeds3	MLP 256–128 · corr loss · 6 ep · lr 0.002 · 3 seeds	0.0262	0.0085	0.0086	0.355
r6_lr2e3_ep5	MLP 256–128 · corr loss · 5 ep · lr 0.002	0.0258	0.0080	0.0081	0.360
r6_wide_lr2e3_ep5	MLP 1024–512–256 · corr loss · 5 ep · lr 0.002 · drop 0.2	0.0257	0.0079	0.0078	0.348

Round 7 · literature ideas

Run	Configuration	corr	BMC	last 200	w/ bench.
r7_bench_pen1	MLP 1024–512–256 · corr loss · 6 ep · lr 0.002 · drop 0.2	0.0188	0.0103	0.0106	0.157
r7_bench_pen03	MLP 1024–512–256 · corr loss · 6 ep · lr 0.002 · drop 0.2	0.0216	0.0102	0.0104	0.217
r7_interp12	MLP 1024–512–256 · corr loss · 6 ep · lr 0.002 · drop 0.2	0.0246	0.0094	0.0094	0.292
r7_bmc_loss	MLP 1024–512–256 · corr loss · 6 ep · lr 0.002 · drop 0.2	0.0248	0.0090	0.0090	0.307
r7_interp08	MLP 1024–512–256 · corr loss · 6 ep · lr 0.002 · drop 0.2	0.0262	0.0076	0.0076	0.367
r7_ender60	MLP 1024–512–256 · corr loss · 6 ep · lr 0.002 · drop 0.2	0.0256	0.0069	0.0063	0.346

Round 8 · confirmation

Run	Configuration	corr	BMC	last 200	w/ bench.
r8_pen1_b60_seeds3	MLP 1024–512–256 · corr loss · 6 ep · lr 0.002 · drop 0.2 · 3 seeds	0.0184	0.0081	0.0080	0.191
r8_pen03_b60_seeds3	MLP 1024–512–256 · corr loss · 6 ep · lr 0.002 · drop 0.2 · 3 seeds	0.0215	0.0081	0.0079	0.249
r8_pen03_b20_seeds3	MLP 1024–512–256 · corr loss · 6 ep · lr 0.002 · drop 0.2 · 3 seeds	0.0220	0.0077	0.0075	0.265

Scale · unseen eras

Run	Configuration	corr	BMC	last 200	w/ bench.
s1_wide_ep3_seeds3	MLP 1024–512–256 · corr loss · 3 ep · lr 0.002 · drop 0.2 · 3 seeds	0.0262	0.0083	0.0095	0.365
s1_wide_ep4_seeds3	MLP 1024–512–256 · corr loss · 4 ep · lr 0.002 · drop 0.2 · 3 seeds	0.0237	0.0074	0.0082	0.330
s1_narrow_ep6_seeds3	MLP 256–128 · corr loss · 6 ep · lr 0.002 · 3 seeds	0.0220	0.0068	0.0067	0.308
s1_wide_ep6_seeds3	MLP 1024–512–256 · corr loss · 6 ep · lr 0.002 · drop 0.2 · 3 seeds	0.0206	0.0065	0.0056	0.283
s1_wide_mse_ep8	MLP 1024–512–256 · MSE · 8 ep · lr 0.001 · drop 0.2	0.0164	0.0049	0.0027	0.233

s2

Run	Configuration	corr	BMC	last 200	w/ bench.
s2_pen03_b60_ep3_seeds3	MLP 1024–512–256 · corr loss · 3 ep · lr 0.002 · drop 0.2 · 3 seeds	0.0208	0.0074	0.0085	0.255